Ethan Hosier

+44 7476 133726 | ethanjhosier@gmail.com | github.com/EthanHosier

Education

Imperial College London — London, UK
MEng Computing - First Class Honours — *Oct. 2022 – June 2026*

Experience

Qube Research & Technologies (QRT) | *Kotlin, Python, AWS* — Jun 2026–Present
Software Engineer, Intern

- Architected a lock-free staging and compaction pipeline for concurrent trade-relationship graph updates, using domain-specific precedence rules to resolve read-compute-write races while preserving graph consistency.
- Optimised graph compaction with parallel workers and no-op filtering and eliminated index bloat through automated staging table truncation, cutting latency from 39.25 ms to 0.047 ms.
- Redesigned a market-risk scenario engine's temporal cache, eliminating recurring OOMs by reducing memory usage from approximately 70 GB across 250,000 entries to under 200 MB across 146 entries and enabling full historical backfills.
- Diagnosed an unbounded backpressure gap in an external library and patched its internal streaming buffer to cap queued data, preventing multi-GB memory growth and OOMs during S3 exports.

Palantir | *Python, Spark, TypeScript* — Apr–Sep 2025
Forward Deployed Software Engineer, Intern — *London, UK*

- Re-architected a Spark entity-resolution pipeline linking billions of transactions across millions of customers, parallelising customer clustering and transaction attribution, then joining results to cut execution time by 71%.
- Designed incremental graph processing that consumed only new records, identified affected customer and transaction clusters, and selectively recomputed them instead of rebuilding the entire graph.
- Optimised distributed execution through broadcast joins, dataset indexing and persisted intermediate results, eliminating large shuffles and redundant Spark recomputation.

Revolut | *Java, TypeScript, React.js, PostgreSQL, Google Cloud Platform* — Jul–Sep 2024
Software Engineer, Intern — *London, UK*

- Replaced repeated queries and full recomputation of global product configurations with an in-process cache incrementally updated from a PostgreSQL event stream, reducing query latency by 92%.
- Reduced production pricing-rule execution time by 24% through caching and parallelising independent calculations.
- Built a dashboard for product and pricing-rule configurations, used daily by the CEO and leadership team, eliminating the need for manual SQL queries.

Projects

PintOS Operating System Kernel | *C*

- Implemented demand paging with clock-based page replacement, swapping, stack growth and memory-mapped files
- Added argument passing and process creation/wait handling for user programs, and implemented system calls for process control and I/O
- Implemented priority-based kernel thread scheduling with priority donation and a multilevel feedback queue

Refactoring Analysis & Synthesis Tool | *Kotlin, Java, TypeScript*

- Processed IDE events into debounced edit snapshots, reconstructing refactoring sessions as replayable Git histories.
- Synthesised alternative refactoring sequences using dependency graphs and prefix-trie search.
- Achieved 83.9% category-level accuracy across four detection categories on 45 controlled evaluation sessions.

WACC Compiler | *Scala*

- Implemented WACC parsing and semantic checks in Scala, including type checking and scoped symbol resolution
- Generated x86-64 assembly for functions, arrays, pairs, and control flow, with stack and register allocation
- Implemented constant folding, function inlining, dead code elimination and peephole optimisations, reducing execution time by 12%

Technical Skills

Rust, C/C++, Kotlin, Java, Python, Go, TypeScript, AWS, React.js, Terraform